title: "寫一個 readme.md 內容包含 demo網址 https://bibaisafish.github.io/DRL_HW1/"

exported: 2026-03-11 02:29:07

url: https://gemini.google.com/app/00b7a24f82110e6f?hl=zh-TW

messages: 16

寫一個 readme.md

內容包含 demo網址 https://bibaisafish.github.io/DRL_HW1/

You

作業內容 HW1-1: 網格地圖開發 目標：開發一個大小為 nxn 的網格地圖，允許用戶指定維度 'n'（範圍從 5 到9），使用 Flask 網頁應用程式。 功能要求： 用戶可以透過滑鼠點擊指定起始單元格（點擊後顯示為綠色）和結束單元格（點擊後顯示為紅色）。 用戶可以設定 n-2 個障礙物，透過滑鼠點擊將這些單元格變為灰色。 HW1-2: 策略顯示與價值評估 目標：顯示每個單元格的隨機生成行動（上下左右箭頭）作為策略。 功能要求：使用策略評估推導出每個狀態的價值 V(s)。

評分標準 HW1-1 網格地圖開發 60% 網格地圖功能完整性 30% 使用者界面友好性 15% 程式碼結構與可讀性 10% 網頁操作流暢度 5%

HW1-2 策略顯示與價值評估 40% 隨機生成行動顯示功能 20%。 策略評估的正確性 15% 程式碼結構與可讀性 5%

Gemini

這是一個非常經典的強化學習（Reinforcement Learning）基礎作業，結合了前端網頁互動與後端演算法。你的目標是建立一個「Grid World」環境，讓使用者設定初始狀態，然後在後端執行**策略評估（Policy Evaluation）**。

為了解決這個作業，我建議將任務拆解為「前端介面（HW1-1）」與「後端演算法（HW1-2）」兩個部分。以下是實作的架構指南與核心邏輯：

🛠️ 系統架構建議

1. **前端 (HTML / CSS / JavaScript)**：負責繪製 $n \times n$ 的網格，並處理使用者的滑鼠點擊事件來切換單元格狀態（起點、終點、障礙物）。
2. **後端 (Python / Flask)**：接收前端傳來的網格狀態（維度 n、起點、終點、障礙物位置），生成隨機策略（Random Policy），並計算出每個狀態的價值 $V(s)$。

🟢 HW1-1: 網格地圖開發 (前端與互動)

這部分的重點在於**動態生成網格**與**狀態管理**。

- **表單輸入**：建立一個輸入框限制輸入值為 5 到 9，並有一個「Generate Square」按鈕。
- **動態生成 (JavaScript)**：點擊生成後，利用 JavaScript 在畫面上畫出一個 HTML Table 或 CSS Grid。
- **點擊邏輯 (State Machine)**： 你需要一個陣列或字典來記錄每個格子的狀態（例如 `0: 空白, 1: 起點, 2: 終點, 3: 障礙物`）。 點擊事件的邏輯應該設計為：
  1. 第一次點擊：設為**起點**（CSS 背景改為 `green`）。
  2. 第二次點擊：設為**終點**（CSS 背景改為 `red`）。
  3. 接下來的 $n-2$ 次點擊：設為**障礙物**（CSS 背景改為 `grey`）。

🧠 HW1-2: 策略顯示與價值評估 (後端 Flask 與演算法)

當使用者在前端設定好地圖並按下「計算價值」的按鈕後，利用 AJAX 或 Form Submit 將資料傳給 Flask 後端。

1. 隨機生成行動 (Random Policy)

每個非障礙物、非終點的格子，都需要被賦予一個策略 $\pi(a|s)$。 因為題目要求隨機生成行動，這代表在每個狀態 s 採取上、下、左、右四個方向的機率皆為相等： $$\pi(\text{up}|s) = \pi(\text{down}|s) = \pi(\text{left}|s) = \pi(\text{right}|s) = 0.25$$

2. 策略評估 (Policy Evaluation)

你需要實作 Bellman Expectation Equation 來迭代計算每個狀態的價值 $V(s)$，直到數值收斂。公式如下：

$$V_{k+1}(s) = \sum_{a} \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')]$$

實作演算法的注意事項：

- **初始化**：建立一個 $n \times n$ 的矩陣，將所有狀態的初始值 $V_0(s)$ 設為 0。
- **環境動態 (Transitions)**：
  - 如果朝某個方向移動會撞到「邊界」或「障礙物」，則狀態保持在原地 $s' = s$。
  - 如果到達「終點」，該狀態的價值固定為 0（或依據你課堂的設定給予 Reward），並且不再進行轉移。

- **獎勵 (Reward, r)**：題目沒有明說，但通常 Grid World 每走一步的 Reward 會設為 `-1`，這代表我們希望 Agent 越快走到終點越好。
- **折扣因子 (Discount Factor, γ)**：通常設為 `1.0` 或 `0.9`。

Python 虛擬碼範例：

```
def policy_evaluation(grid_size, start, end, obstacles, theta=0.0001, gamma=1.0):
    # 初始化 V(s) 為 0 的 n x n 矩陣
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]

    while True:
        delta = 0
        # 遍歷所有格子
        for i in range(grid_size):
            for j in range(grid_size):
                if (i, j) == end or (i, j) in obstacles:
                    continue # 終點和障礙物不更新 V 值

                v_old = V[i][j]
                v_new = 0

                # 計算四個方向的期望值 (機率皆為 0.25)
                for action in [(0,1), (0,-1), (1,0), (-1,0)]: # 右, 左, 下, 上
                    next_i, next_j = i + action[0], j + action[1]

                    # 邊界或障礙物判定
                    if next_i < 0 or next_i >= grid_size or next_j < 0 or next_j >= grid_size or (next_i, next_j) in obstacles:
                        next_i, next_j = i, j # 留在原地

                    reward = -1 # 假設每走一步 reward 為 -1
                    v_new += 0.25 * (reward + gamma * V[next_i][next_j])

                V[i][j] = v_new
                delta = max(delta, abs(v_old - v_new))

        if delta < theta: # 收斂條件
            break

    return V
```

這個作業的範圍涵蓋了前後端開發以及強化學習演算法的實作。為了能最有效率地幫助你推進進度：

你想先從哪一個部分開始著手？ 我可以先為你撰寫 **HW1-1 的 HTML/JavaScript 網格互動程式碼**，或是提供 **Flask 後端架設與完整 Policy Evaluation 演算法的 Python 程式碼**。

You

先從 index.html 開始生成

Gemini

沒問題！我們先把前端的互動介面與網格地圖（HW1-1）搭建起來。

這裡我為你準備了一個整合了 HTML、CSS 和 JavaScript 的單一 `index.html` 檔案。這支程式會處理表單輸入驗證、動態生成網格，並且實作了「點擊設定起點 $\rightarrow$ 點點設定終點 $\rightarrow$ 點擊設定障礙物」的狀態切換邏輯。

你可以直接建立一個名為 `index.html` 的檔案，然後把以下程式碼貼進去，並在瀏覽器中打開測試：

```
<!DOCTYPE html>
<html lang="zh-TW">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Grid World Generator</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 40px;
            line-height: 1.6;
        }
        .controls {
            margin-bottom: 20px;
        }
        /* 網格容器 */
        #gridContainer {
            display: inline-block;
            border: 2px solid #000;
            margin-top: 20px;
        }
        .grid-row {
            display: flex;
        }
        .grid-cell {
            width: 50px;
            height: 50px;
            border: 1px solid #333;
```

```
            display: flex;
            align-items: center;
            justify-content: center;
            cursor: pointer;
            box-sizing: border-box;
            font-size: 16px;
            user-select: none; /* 防止反白文字 */
        }
        /* 各種單元格狀態的顏色 */
        .cell-start { background-color: green; color: white; font-weight: bold; }
        .cell-end { background-color: red; color: white; font-weight: bold; }
        .cell-obstacle { background-color: grey; color: transparent; } /* 障礙物可以隱藏數字 */

        #instruction {
            font-size: 18px;
            font-weight: bold;
            color: #333;
            margin-top: 10px;
        }
        #calculateBtn {
            margin-top: 20px;
            padding: 10px 20px;
            font-size: 16px;
            background-color: #007BFF;
            color: white;
            border: none;
            cursor: pointer;
            display: none; /* 初始隱藏，設定完才顯示 */
        }
        #calculateBtn:hover {
            background-color: #0056b3;
        }
    </style>
</head>
<body>

    <h1>Generate n x n Square</h1>

    <div class="controls">
        <label for="gridSize">Enter a number between 5 and 9:</label>
        <input type="number" id="gridSize" min="5" max="9" value="5">
        <button onclick="generateGrid()">Generate Square</button>
    </div>

    <h2 id="gridTitle" style="display: none;"></h2>
    <div id="instruction"></div>

    <div id="gridContainer"></div>

    <button id="calculateBtn" onclick="submitToBackend()">計算策略與價值 (Submit to Flask)</button>

    <script>
        // 狀態管理變數
        let n = 0;
        let clickState = 0; // 0: 等待設定起點, 1: 等待設定終點, 2: 等待設定障礙物, 3: 設定完成
        let obstaclesLeft = 0;

        // 儲存地圖資訊，準備傳給後端
        let mapData = {
            size: 0,
            start: null,
            end: null,
            obstacles: []
        };

        function generateGrid() {
            const inputVal = document.getElementById('gridSize').value;
            n = parseInt(inputVal);

            // 驗證輸入範圍
            if (n < 5 || n > 9) {
                alert("Please enter a number between 5 and 9.");
                return;
            }

            // 重置狀態與資料
            clickState = 0;
            obstaclesLeft = n - 2;
            mapData = { size: n, start: null, end: null, obstacles: [] };

            // 更新 UI 標題與提示
            document.getElementById('gridTitle').style.display = 'block';
            document.getElementById('gridTitle').innerText = `${n} x ${n} Square:`;
            document.getElementById('calculateBtn').style.display = 'none';
            updateInstruction();

            // 生成網格 DOM
            const container = document.getElementById('gridContainer');
            container.innerHTML = ''; // 清空舊網格
```

```
        let cellNumber = 1;
        for (let i = 0; i < n; i++) {
            const rowDiv = document.createElement('div');
            rowDiv.className = 'grid-row';
            for (let j = 0; j < n; j++) {
                const cellDiv = document.createElement('div');
                cellDiv.className = 'grid-cell';
                cellDiv.innerText = cellNumber++;
                // 綁定點擊事件，並傳入座標 (i, j)
                cellDiv.onclick = (e) => handleCellClick(e.target, i, j);
                rowDiv.appendChild(cellDiv);
            }
            container.appendChild(rowDiv);
        }
    }

    function handleCellClick(cellElement, i, j) {
        // 如果這個格子已經被設定過了，就不做任何反應
        if (cellElement.classList.contains('cell-start') ||
            cellElement.classList.contains('cell-end') ||
            cellElement.classList.contains('cell-obstacle')) {
            return;
        }

        if (clickState === 0) {
            // 設定起點 (綠色)
            cellElement.classList.add('cell-start');
            mapData.start = [i, j];
            clickState = 1;
            updateInstruction();
        }
        else if (clickState === 1) {
            // 設定終點 (紅色)
            cellElement.classList.add('cell-end');
            mapData.end = [i, j];
            clickState = 2;
            updateInstruction();
        }
        else if (clickState === 2) {
            // 設定障礙物 (灰色)
            cellElement.classList.add('cell-obstacle');
            mapData.obstacles.push([i, j]);
            obstaclesLeft--;
            updateInstruction();

            // 障礙物設定完畢
            if (obstaclesLeft === 0) {
                clickState = 3;
                document.getElementById('calculateBtn').style.display = 'block'; // 顯示提交按鈕
            }
        }
    }

    function updateInstruction() {
        const inst = document.getElementById('instruction');
        if (clickState === 0) {
            inst.innerText = "1. Click on a cell to set up the start grid as green.";
            inst.style.color = "green";
        } else if (clickState === 1) {
            inst.innerText = "2. Click on a cell to set up the end grid as red.";
            inst.style.color = "red";
        } else if (clickState === 2) {
            inst.innerText = `3. Click on cells to set up obstacles (Grey). ${obstaclesLeft} left.`;
            inst.style.color = "grey";
        } else if (clickState === 3) {
            inst.innerText = "Setup complete! Ready to calculate.";
            inst.style.color = "blue";
        }
    }

    function submitToBackend() {
        // 這個函數是用來將 mapData 透過 AJAX 傳給 Flask 的
        // 目前先用 console.log 讓你看看資料結構
        console.log("準備傳送給後端的資料格式：", mapData);
        alert("請打開 F12 開發者工具的 Console 查看資料格式。下一步我們將把這些資料傳給 Flask！");
    }
</script>

</body>
</html>
```

💡 程式碼亮點說明

1. **狀態機 (State Machine) 管理**：我使用了 `clickState` 變數來控制使用者的點擊流程（起點 $\rightarrow$ 終點 $\rightarrow$ 障礙物）。這可以確保使用者不會在設定起點前就先設定障礙物。
2. **資料結構 `mapData`**：在使用者點擊的過程中，程式會自動將座標 `[i, j]` 記錄在 `mapData` 裡面。這正是我們之後發送 AJAX 請求給 Flask 時，所需的完美 JSON 格式。
3. **動態提示**：隨著步驟的進行，畫面上方的提示文字與顏色會動態改變，完全符合「使用者界面友好性 15%」的評分標準。

打開這個 HTML 檔案玩玩看，確認互動邏輯是不是如你預期。

接下來，你需要我為你撰寫後端的 `app.py` (Flask) 以及接收資料、進行 Policy Evaluation 的演算法部分嗎？

You

灰色格子的數字不見了

Gemini

啊！好眼力。這是我在之前的程式碼裡特意加上去的，不過仔細對照你一開始提供的作業參考圖，灰色的格子確實應該要保留原本的黑色數字才對！

要解決這個問題非常簡單，只需要修改 HTML 檔案中 `<style>` 裡的一行 CSS 即可：

請找到原本的這行程式碼：

```
.cell-obstacle { background-color: grey; color: transparent; } /* 障礙物可以隱藏數字 */
```

將它改成這樣（把 `color` 設為 `black` 或者直接刪掉 `color` 屬性）：

```
.cell-obstacle { background-color: grey; color: black; }
```

存檔並重新整理網頁後，灰色格子裡的數字就會乖乖出現了！

修改完這個小細節後，前端的畫面應該就跟作業的要求一模一樣了。**需要我接著為你產出 Flask 後端 (`app.py`) 的程式碼，把資料傳過去並計算 Policy 和 Value 嗎？**

You

我想要讓他 DEFAULT 設5，一點開就先生成 5x5 grid

Gemini

這個想法很好！這樣使用者一打開網頁就能直接開始點擊，體驗會流暢很多。

要達成這個效果非常簡單，我們只需要在網頁載入完成時，自動呼叫一次 `generateGrid()` 函數就可以了。

請在你的 `index.html` 檔案中，找到 `<script>` 標籤的最底部（也就是 `</script>` 的上一行），加入這段程式碼：

```
        // ... (前面的程式碼保留) ...

        function submitToBackend() {
            console.log("準備傳送給後端的資料格式：", mapData);
            alert("請打開 F12 開發者工具的 Console 查看資料格式。下一步我們將把這些資料傳給 Flask！");
        }

        // 加入這段：當網頁載入完成後，自動觸發生成 5x5 網格
        window.onload = function() {
            generateGrid();
        };
    </script>
```

加上這段之後，只要重新整理網頁，畫面就會預設幫你畫好一個 5x5 的網格，並且直接進入「等待設定起點」的狀態了！

這樣前端的 HW1-1 體驗應該就非常完整了。**準備好進入 Flask 後端與演算法的部分了嗎？**

You

part 1 的前端畫面好了 換 part 2

Gemini

太棒了！前端的基礎已經打好，現在我們進入核心的 **Part 2：Flask 後端與強化學習演算法 (Policy Evaluation)**。

為了完成這部分，我們需要做兩件事：

1. 建立一個 Python 後端 (`app.py`) 來接收前端傳來的網格資料，並執行 **Policy Evaluation (策略評估)** 演算法來算出每個格子的價值 $V(s)$，同時推導出每個格子的策略箭頭。
2. 更新前端的 JavaScript，讓它能把資料傳給後端，並將算出來的結果畫成兩個新的網格（Value Matrix 和 Policy Matrix）。

🟢 步驟 1：建立 Flask 後端 (`app.py`)

請在與 `index.html` 同一個資料夾下，建立一個名為 `app.py` 的檔案。

這個程式會建立一個 `/calculate` 的 API 接口。它會對**隨機策略 (Random Policy, 每個方向機率 25%)** 進行評估，算出價值矩陣後，再找出每個格子的**貪婪策略 (Greedy Policy)** 來決定箭頭方向（這完全符合你上傳的第二張參考圖的效果）。

將以下程式碼貼入 `app.py`：

```
from flask import Flask, request, jsonify
from flask_cors import CORS
```

```python
app = Flask(__name__)
CORS(app) # 允許前端跨域請求

def policy_evaluation(grid_size, start, end, obstacles, theta=1e-4, gamma=1.0):
    # 初始化 Value Matrix (V)
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]

    # 定義動作：上, 下, 左, 右 (row_change, col_change)
    actions = {
        'up': (-1, 0),
        'down': (1, 0),
        'left': (0, -1),
        'right': (0, 1)
    }

    # 策略評估 (Policy Evaluation) - 評估隨機策略 (每個動作機率 0.25)
    while True:
        delta = 0
        new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]

        for r in range(grid_size):
            for c in range(grid_size):
                state = [r, c]

                # 終點和障礙物的 V 值保持為 0
                if state == end or state in obstacles:
                    continue

                v_expected = 0
                for action_name, (dr, dc) in actions.items():
                    next_r, next_c = r + dr, c + dc

                    # 邊界與障礙物判定 (如果撞牆或撞障礙物，留在原地)
                    if next_r < 0 or next_r >= grid_size or next_c < 0 or next_c >= grid_size or [next_r, next_c] in obstacles:
                        next_r, next_c = r, c

                    # 獎勵設定：每走一步 reward = -1
                    reward = -1

                    # 隨機策略：每個動作機率 0.25
                    v_expected += 0.25 * (reward + gamma * V[next_r][next_c])

                new_V[r][c] = v_expected
                delta = max(delta, abs(v_expected - V[r][c]))

        V = new_V
        if delta < theta:
            break

    # 推導策略矩陣 (Policy Matrix) - 找出可以得到最大 V 值的方向 (Greedy Policy)
    policy = [[[] for _ in range(grid_size)] for _ in range(grid_size)]
    for r in range(grid_size):
        for c in range(grid_size):
            state = [r, c]
            if state == end or state in obstacles:
                continue

            max_val = -float('inf')
            best_actions = []

            for action_name, (dr, dc) in actions.items():
                next_r, next_c = r + dr, c + dc
                if next_r < 0 or next_r >= grid_size or next_c < 0 or next_c >= grid_size or [next_r, next_c] in obstacles:
                    next_r, next_c = r, c

                val = -1 + gamma * V[next_r][next_c]

                # 處理浮點數誤差，找出最佳行動
                if val > max_val + 1e-6:
                    max_val = val
                    best_actions = [action_name]
                elif abs(val - max_val) <= 1e-6:
                    best_actions.append(action_name)

            policy[r][c] = best_actions

    return V, policy

@app.route('/calculate', methods=['POST'])
def calculate():
    data = request.json
    grid_size = data['size']
    start = data['start']
    end = data['end']
    obstacles = data['obstacles']

    # 執行演算法
    V, policy = policy_evaluation(grid_size, start, end, obstacles)
```

```
    # 將浮點數四捨五入到小數點後兩位，方便前端顯示
    V_rounded = [[round(val, 2) for val in row] for row in V]

    return jsonify({
        'value_matrix': V_rounded,
        'policy_matrix': policy
    })

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

注意： 執行前，請確保你有安裝 Flask 和 Flask-CORS。可以在終端機執行：`pip install flask flask-cors`

🟢 步驟 2：更新前端 `index.html` 的串接邏輯

現在我們需要修改 `index.html` 中的 `submitToBackend()` 函數，讓它能把資料送到 Flask，並將回傳的結果畫成圖表。

請在 `index.html` 的 `<style>` 中**加入**這段 CSS（用來排版結果矩陣）：

```
        .result-container {
            display: flex;
            gap: 40px;
            margin-top: 30px;
        }
        .matrix-wrapper {
            display: flex;
            flex-direction: column;
            align-items: center;
        }
        .matrix-wrapper h3 { margin-bottom: 10px; }
        .arrow { font-size: 20px; font-weight: bold; }
```

接著，在 `<div id="gridContainer"></div>` 和 `<button id="calculateBtn">` 之後，**加入**用來顯示結果的 HTML 區塊：

```
    <div id="results" class="result-container" style="display: none;">
        <div class="matrix-wrapper">
            <h3>Value Matrix</h3>
            <div id="valueGrid"></div>
        </div>
        <div class="matrix-wrapper">
            <h3>Policy Matrix</h3>
            <div id="policyGrid"></div>
        </div>
    </div>
```

最後，**覆蓋**掉原本的 `submitToBackend()` 函數：

```
        async function submitToBackend() {
            try {
                // 發送請求給 Flask
                const response = await fetch('http://127.0.0.1:5000/calculate', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify(mapData)
                });

                const result = await response.json();

                // 顯示結果區塊
                document.getElementById('results').style.display = 'flex';

                // 繪製 Value Matrix 和 Policy Matrix
                renderResultGrid('valueGrid', result.value_matrix, 'value');
                renderResultGrid('policyGrid', result.policy_matrix, 'policy');

            } catch (error) {
                console.error("Error:", error);
                alert("無法連接到後端，請確認 Flask 伺服器 (app.py) 是否正在執行！");
            }
        }

        // 用來繪製結果矩陣的輔助函數
        function renderResultGrid(containerId, matrixData, type) {
            const container = document.getElementById(containerId);
            container.innerHTML = '';

            // 建立箭頭的映射字典
            const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

            for (let i = 0; i < n; i++) {
                const rowDiv = document.createElement('div');
                rowDiv.className = 'grid-row';
                for (let j = 0; j < n; j++) {
                    const cellDiv = document.createElement('div');
                    cellDiv.className = 'grid-cell';

                    // 判斷是否為特殊格子（終點或障礙物）
                    const isEnd = (mapData.end[0] === i && mapData.end[1] === j);
                    const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j);
```

```
                if (isEnd) {
                    cellDiv.classList.add('cell-end');
                    cellDiv.innerText = 'END';
                } else if (isObstacle) {
                    cellDiv.classList.add('cell-obstacle');
                } else {
                    // 填入 Value 或 Policy
                    if (type === 'value') {
                        cellDiv.innerText = matrixData[i][j];
                    } else if (type === 'policy') {
                        // 將 action 陣列轉換成箭頭符號
                        const actionList = matrixData[i][j];
                        cellDiv.innerHTML = `<span class="arrow">${actionList.map(a => arrows[a]).join('')}</span>`;
                    }
                }
                rowDiv.appendChild(cellDiv);
            }
            container.appendChild(rowDiv);
        }
    }
```

🚀 如何執行測試？

1. 打開你的終端機 (Terminal / 命令提示字元)。
2. 進入到這兩個檔案所在的資料夾。
3. 執行 Flask 伺服器：

   ```
   python app.py
   ```

   (你應該會看到 `Running on http://127.0.0.1:5000/` 的提示)
4. 直接用瀏覽器打開你的 `index.html`。
5. 在網頁上依序點擊設定起點、終點、障礙物，然後按下 **"計算策略與價值"**。

這時候，畫面下方就會出現像你作業參考圖一樣的 **Value Matrix (數字)** 與 **Policy Matrix (箭頭)** 了！

你可以先跑跑看，確認運作是否順利。如果需要調整 UI 顏色或是數值計算的細節（例如 Reward 數值的設定），隨時告訴我！

You

幫我改成用 js, 並將 index.html 拆成三個檔案 分別是 html , css, js 檔，並把part 2 寫進 js 中

Gemini

沒問題！完全使用純前端（JavaScript）來執行強化學習演算法也是一個很棒的做法，這樣就不需要架設伺服器，只要打開瀏覽器就能直接算出結果。

我已經幫你把程式碼拆分成三個乾淨的檔案：`index.html`、`style.css` 和 `script.js`，並且將原本 Python 的 Policy Evaluation 演算法完全翻譯成 JavaScript 寫入 `script.js` 中。

請在同一個資料夾下建立這三個檔案，並將以下程式碼分別貼入：

1. `index.html` (網頁結構)

這個檔案現在非常乾淨，只負責 HTML 結構，並引入了 CSS 和 JS 檔案。

```
<!DOCTYPE html>
<html lang="zh-TW">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Grid World Generator</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <h1>Generate n x n Square</h1>

    <div class="controls">
        <label for="gridSize">Enter a number between 5 and 9:</label>
        <input type="number" id="gridSize" min="5" max="9" value="5">
        <button onclick="generateGrid()">Generate Square</button>
    </div>

    <h2 id="gridTitle" style="display: none;"></h2>
    <div id="instruction"></div>

    <div id="gridContainer"></div>

    <button id="calculateBtn" onclick="calculatePolicy()">計算策略與價值 (Calculate)</button>

    <div id="results" class="result-container" style="display: none;">
        <div class="matrix-wrapper">
            <h3>Value Matrix</h3>
            <div id="valueGrid"></div>
        </div>
        <div class="matrix-wrapper">
            <h3>Policy Matrix</h3>
```

```
            <div id="policyGrid"></div>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

---

2. `style.css` (外觀樣式)

負責所有的排版與顏色設定。

```
body {
    font-family: Arial, sans-serif;
    margin: 40px;
    line-height: 1.6;
}

.controls {
    margin-bottom: 20px;
}

/* 網格容器 */
#gridContainer, #valueGrid, #policyGrid {
    display: inline-block;
    border: 2px solid #000;
    margin-top: 20px;
    background-color: #fff;
}

.grid-row {
    display: flex;
}

.grid-cell {
    width: 60px;
    height: 60px;
    border: 1px solid #333;
    display: flex;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    box-sizing: border-box;
    font-size: 16px;
    user-select: none; /* 防止反白文字 */
    text-align: center;
}

/* 各種單元格狀態的顏色 */
.cell-start { background-color: green; color: white; font-weight: bold; }
.cell-end { background-color: red; color: white; font-weight: bold; }
.cell-obstacle { background-color: grey; color: black; } /* 顯示黑色數字 */

#instruction {
    font-size: 18px;
    font-weight: bold;
    color: #333;
    margin-top: 10px;
}

#calculateBtn {
    margin-top: 20px;
    padding: 10px 20px;
    font-size: 16px;
    background-color: #007BFF;
    color: white;
    border: none;
    cursor: pointer;
    display: none; /* 初始隱藏，設定完才顯示 */
    border-radius: 4px;
}

#calculateBtn:hover {
    background-color: #0056b3;
}

/* 結果矩陣區塊 */
.result-container {
    display: flex;
    gap: 40px;
    margin-top: 30px;
    flex-wrap: wrap; /* 螢幕不夠寬時自動換行 */
}

.matrix-wrapper {
    display: flex;
    flex-direction: column;
    align-items: center;
}
```

```
.matrix-wrapper h3 {
    margin-bottom: 0px;
}

.arrow {
    font-size: 20px;
    font-weight: bold;
    line-height: 1;
}
```

---

3. `script.js` (互動邏輯與演算法)

這裡包含了 UI 的互動邏輯，以及最核心的 Policy Evaluation 演算法（原本在 Flask 裡的部分）。

```
// 狀態管理變數
let n = 0;
let clickState = 0; // 0: 等待設定起點, 1: 等待設定終點, 2: 等待設定障礙物, 3: 設定完成
let obstaclesLeft = 0;

// 儲存地圖資訊
let mapData = {
    size: 0,
    start: null,
    end: null,
    obstacles: []
};

// 網頁載入完成後自動生成 5x5 網格
window.onload = function() {
    generateGrid();
};

function generateGrid() {
    const inputVal = document.getElementById('gridSize').value;
    n = parseInt(inputVal);

    if (n < 5 || n > 9) {
        alert("Please enter a number between 5 and 9.");
        return;
    }

    // 重置狀態與資料
    clickState = 0;
    obstaclesLeft = n - 2;
    mapData = { size: n, start: null, end: null, obstacles: [] };

    // 更新 UI
    document.getElementById('gridTitle').style.display = 'block';
    document.getElementById('gridTitle').innerText = `${n} x ${n} Square:`;
    document.getElementById('calculateBtn').style.display = 'none';
    document.getElementById('results').style.display = 'none'; // 隱藏之前的結果
    updateInstruction();

    // 生成網格 DOM
    const container = document.getElementById('gridContainer');
    container.innerHTML = '';

    let cellNumber = 1;
    for (let i = 0; i < n; i++) {
        const rowDiv = document.createElement('div');
        rowDiv.className = 'grid-row';
        for (let j = 0; j < n; j++) {
            const cellDiv = document.createElement('div');
            cellDiv.className = 'grid-cell';
            cellDiv.innerText = cellNumber++;
            cellDiv.onclick = (e) => handleCellClick(e.target, i, j);
            rowDiv.appendChild(cellDiv);
        }
        container.appendChild(rowDiv);
    }
}

function handleCellClick(cellElement, i, j) {
    if (cellElement.classList.contains('cell-start') ||
        cellElement.classList.contains('cell-end') ||
        cellElement.classList.contains('cell-obstacle')) {
        return;
    }

    if (clickState === 0) {
        cellElement.classList.add('cell-start');
        mapData.start = [i, j];
        clickState = 1;
        updateInstruction();
    }
    else if (clickState === 1) {
        cellElement.classList.add('cell-end');
        mapData.end = [i, j];
```

```
        clickState = 2;
        updateInstruction();
    }
    else if (clickState === 2) {
        cellElement.classList.add('cell-obstacle');
        mapData.obstacles.push([i, j]);
        obstaclesLeft--;
        updateInstruction();

        if (obstaclesLeft === 0) {
            clickState = 3;
            document.getElementById('calculateBtn').style.display = 'block';
        }
    }
}

function updateInstruction() {
    const inst = document.getElementById('instruction');
    if (clickState === 0) {
        inst.innerText = "1. Click on a cell to set up the start grid as green.";
        inst.style.color = "green";
    } else if (clickState === 1) {
        inst.innerText = "2. Click on a cell to set up the end grid as red.";
        inst.style.color = "red";
    } else if (clickState === 2) {
        inst.innerText = `3. Click on cells to set up obstacles (Grey). ${obstaclesLeft} left.`;
        inst.style.color = "grey";
    } else if (clickState === 3) {
        inst.innerText = "Setup complete! Ready to calculate.";
        inst.style.color = "blue";
    }
}

// === HW1-2: 策略評估演算法 (JavaScript 實作) ===

function calculatePolicy() {
    const gridSize = mapData.size;
    const end = mapData.end;
    const obstacles = mapData.obstacles;

    const theta = 1e-4;
    const gamma = 1.0;

    // 初始化 V 為 0
    let V = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0));
    const actions = {
        'up': [-1, 0], 'down': [1, 0], 'left': [0, -1], 'right': [0, 1]
    };

    // 輔助函數：判斷是否為障礙物或終點
    const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c);
    const isEnd = (r, c) => end[0] === r && end[1] === c;

    // 1. 執行 Policy Evaluation (隨機策略 0.25)
    while (true) {
        let delta = 0;
        let newV = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0));

        for (let r = 0; r < gridSize; r++) {
            for (let c = 0; c < gridSize; c++) {
                if (isEnd(r, c) || isObstacle(r, c)) continue;

                let vExpected = 0;
                for (let action in actions) {
                    let nextR = r + actions[action][0];
                    let nextC = c + actions[action][1];

                    // 撞牆或障礙物則留在原地
                    if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) {
                        nextR = r;
                        nextC = c;
                    }

                    let reward = -1; // 每走一步的 reward
                    vExpected += 0.25 * (reward + gamma * V[nextR][nextC]);
                }
                newV[r][c] = vExpected;
                delta = Math.max(delta, Math.abs(vExpected - V[r][c]));
            }
        }
        V = newV;
        if (delta < theta) break; // 收斂
    }

    // 2. 找出 Greedy Policy (推導箭頭方向)
    let policy = Array.from({ length: gridSize }, () => Array(gridSize).fill([]));

    for (let r = 0; r < gridSize; r++) {
        for (let c = 0; c < gridSize; c++) {
            if (isEnd(r, c) || isObstacle(r, c)) continue;
```

```
            let maxVal = -Infinity;
            let bestActions = [];

            for (let action in actions) {
                let nextR = r + actions[action][0];
                let nextC = c + actions[action][1];

                if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) {
                    nextR = r;
                    nextC = c;
                }

                let val = -1 + gamma * V[nextR][nextC];

                // 處理浮點數誤差
                if (val > maxVal + 1e-6) {
                    maxVal = val;
                    bestActions = [action];
                } else if (Math.abs(val - maxVal) <= 1e-6) {
                    bestActions.push(action); // 多個方向同等好
                }
            }
            policy[r][c] = bestActions;
        }
    }

    // 將 V 的數值四捨五入到小數點後兩位
    let roundedV = V.map(row => row.map(val => Number(val.toFixed(2))));

    // 顯示結果
    document.getElementById('results').style.display = 'flex';
    renderResultGrid('valueGrid', roundedV, 'value');
    renderResultGrid('policyGrid', policy, 'policy');
}

// 繪製結果圖表的函數
function renderResultGrid(containerId, matrixData, type) {
    const container = document.getElementById(containerId);
    container.innerHTML = '';

    const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

    for (let i = 0; i < n; i++) {
        const rowDiv = document.createElement('div');
        rowDiv.className = 'grid-row';
        for (let j = 0; j < n; j++) {
            const cellDiv = document.createElement('div');
            cellDiv.className = 'grid-cell';

            const isEnd = (mapData.end[0] === i && mapData.end[1] === j);
            const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j);

            if (isEnd) {
                cellDiv.classList.add('cell-end');
                // 可自行決定終點要顯示 END 還是空白
            } else if (isObstacle) {
                cellDiv.classList.add('cell-obstacle');
            } else {
                if (type === 'value') {
                    cellDiv.innerText = matrixData[i][j];
                } else if (type === 'policy') {
                    // 將 action 陣列轉換成多個箭頭
                    const actionList = matrixData[i][j];
                    const arrowStr = actionList.map(a => arrows[a]).join('');
                    cellDiv.innerHTML = `<span class="arrow">${arrowStr}</span>`;
                }
            }
            rowDiv.appendChild(cellDiv);
        }
        container.appendChild(rowDiv);
    }
}
```

這樣一來，你的作業就完全變成了輕量級的網頁應用程式，不依賴任何後端伺服器！直接用瀏覽器打開 `index.html` 就可以測試完整的設定、點擊到計算出矩陣結果的流程。

需要我幫忙解釋演算法裡 `V = newV` 收斂邏輯，或者調整箭頭顯示的大小和樣式嗎？

You

// === HW1-2: 策略評估演算法 (JavaScript 實作) ===

function calculatePolicy() { const gridSize = mapData.size; const end = mapData.end; const obstacles = mapData.obstacles; const theta = 1e-4; const gamma = 1.0; // 初始化 V 為 0 let V = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); const actions = { 'up': [-1, 0], 'down': [1, 0], 'left': [0, -1], 'right': [0, 1] };

// 輔助函數：判斷是否為障礙物或終點 const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c); const isEnd = (r, c) => end[0] === r && end[1] === c;

// 1. 執行 Policy Evaluation (隨機策略 0.25) while (true) { let delta = 0; let newV = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); for (let r = 0; r < gridSize; r++) { for (let c = 0; c < gridSize; c++) { if (isEnd(r, c) || isObstacle(r, c)) continue; let vExpected = 0; for (let action in actions) { let nextR = r + actions[action][0]; let nextC = c + actions[action][1]; // 撞牆或障礙物則留在原地 if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) { nextR = r; nextC = c; } let reward = -1; // 每走一步的 reward vExpected += 0.25 * (reward + gamma * V[nextR][nextC]); } newV[r][c] = vExpected; delta = Math.max(delta, Math.abs(vExpected - V[r][c])); } } V = newV; if (delta < theta) break; // 收斂 }

// 2. 找出 Greedy Policy (推導箭頭方向) let policy = Array.from({ length: gridSize }, () => Array(gridSize).fill([])); for (let r = 0; r < gridSize; r++) { for (let c = 0; c < gridSize; c++) { if (isEnd(r, c) || isObstacle(r, c)) continue; let maxVal = -Infinity; let bestActions = []; for (let action in actions) { let nextR = r + actions[action][0]; let nextC = c + actions[action][1]; if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) { nextR = r; nextC = c; } let val = -1 + gamma * V[nextR][nextC]; // 處理浮點數誤差 if (val > maxVal + 1e-6) { maxVal = val; bestActions = [action]; } else if (Math.abs(val - maxVal) <= 1e-6) { bestActions.push(action); // 多個方向同等好 } } policy[r][c] = bestActions; } }

// 將 V 的數值四捨五入到小數點後兩位 let roundedV = V.map(row => row.map(val => Number(val.toFixed(2))));

// 顯示結果 document.getElementById('results').style.display = 'flex'; renderResultGrid('valueGrid', roundedV, 'value'); renderResultGrid('policyGrid', policy, 'policy'); }

// 繪製結果圖表的函數 function renderResultGrid(containerId, matrixData, type) { const container = document.getElementById(containerId); container.innerHTML = ''; const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

for (let i = 0; i < n; i++) { const rowDiv = document.createElement('div'); rowDiv.className = 'grid-row'; for (let j = 0; j < n; j++) { const cellDiv = document.createElement('div'); cellDiv.className = 'grid-cell'; const isEnd = (mapData.end[0] === i && mapData.end[1] === j); const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j); if (isEnd) { cellDiv.classList.add('cell-end'); // 可自行決定終點要顯示 END 還是空白 } else if (isObstacle) { cellDiv.classList.add('cell-obstacle'); } else { if (type === 'value') { cellDiv.innerText = matrixData[i][j]; } else if (type === 'policy') { // 將 action 陣列轉換成多個箭頭 const actionList = matrixData[i][j]; const arrowStr = actionList.map(a => arrows[a]).join(''); cellDiv.innerHTML = ${arrowStr}; } } rowDiv.appendChild(cellDiv); } container.appendChild(rowDiv); } }

解釋這段的目的與程式碼內容// === HW1-2: 策略評估演算法 (JavaScript 實作) ===

function calculatePolicy() { const gridSize = mapData.size; const end = mapData.end; const obstacles = mapData.obstacles; const theta = 1e-4; const gamma = 1.0; // 初始化 V 為 0 let V = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); const actions = { 'up': [-1, 0], 'down': [1, 0], 'left': [0, -1], 'right': [0, 1] };

// 輔助函數：判斷是否為障礙物或終點 const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c); const isEnd = (r, c) => end[0] === r && end[1] === c;

// 1. 執行 Policy Evaluation (隨機策略 0.25) while (true) { let delta = 0; let newV = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); for (let r = 0; r < gridSize; r++) { for (let c = 0; c < gridSize; c++) { if (isEnd(r, c) || isObstacle(r, c)) continue; let vExpected = 0; for (let action in actions) { let nextR = r + actions[action][0]; let nextC = c + actions[action][1]; // 撞牆或障礙物則留在原地 if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) { nextR = r; nextC = c; } let reward = -1; // 每走一步的 reward vExpected += 0.25 * (reward + gamma * V[nextR][nextC]); } newV[r][c] = vExpected; delta = Math.max(delta, Math.abs(vExpected - V[r][c])); } } V = newV; if (delta < theta) break; // 收斂 }

// 2. 找出 Greedy Policy (推導箭頭方向) let policy = Array.from({ length: gridSize }, () => Array(gridSize).fill([])); for (let r = 0; r < gridSize; r++) { for (let c = 0; c < gridSize; c++) { if (isEnd(r, c) || isObstacle(r, c)) continue; let maxVal = -Infinity; let bestActions = []; for (let action in actions) { let nextR = r + actions[action][0]; let nextC = c + actions[action][1]; if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) { nextR = r; nextC = c; } let val = -1 + gamma * V[nextR][nextC]; // 處理浮點數誤差 if (val > maxVal + 1e-6) { maxVal = val; bestActions = [action]; } else if (Math.abs(val - maxVal) <= 1e-6) { bestActions.push(action); // 多個方向同等好 } } policy[r][c] = bestActions; } }

// 將 V 的數值四捨五入到小數點後兩位 let roundedV = V.map(row => row.map(val => Number(val.toFixed(2))));

// 顯示結果 document.getElementById('results').style.display = 'flex'; renderResultGrid('valueGrid', roundedV, 'value'); renderResultGrid('policyGrid', policy, 'policy'); }

// 繪製結果圖表的函數 function renderResultGrid(containerId, matrixData, type) { const container = document.getElementById(containerId); container.innerHTML = ''; const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

for (let i = 0; i < n; i++) { const rowDiv = document.createElement('div'); rowDiv.className = 'grid-row'; for (let j = 0; j < n; j++) { const cellDiv = document.createElement('div'); cellDiv.className = 'grid-cell'; const isEnd = (mapData.end[0] === i && mapData.end[1] === j); const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j); if (isEnd) { cellDiv.classList.add('cell-end'); // 可自行決定終點要顯示 END 還是空白 } else if (isObstacle) { cellDiv.classList.add('cell-obstacle'); } else { if (type === 'value') { cellDiv.innerText = matrixData[i][j]; } else if (type === 'policy') { // 將 action 陣列轉換成多個箭頭 const actionList = matrixData[i][j]; const arrowStr = actionList.map(a => arrows[a]).join(''); cellDiv.innerHTML = ${arrowStr}; } } rowDiv.appendChild(cellDiv); } container.appendChild(rowDiv); } }

解釋這段的目的與程式碼內容 // === HW1-2: 策略評估演算法 (JavaScript 實作) ===

function calculatePolicy() { const gridSize = mapData.size; const end = mapData.end; const obstacles = mapData.obstacles; const theta = 1e-4; const gamma = 1.0; // 初始化 V 為 0 let V = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); const actions = { 'up': [-1, 0], 'down': [1, 0], 'left': [0, -1], 'right': [0, 1] };

// 輔助函數：判斷是否為障礙物或終點 const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c); const isEnd = (r, c) => end[0] === r && end[1] === c;

// 1. 執行 Policy Evaluation (隨機策略 0.25) while (true) { let delta = 0; let newV = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); for (let r = 0; r < gridSize; r++) { for (let c = 0; c < gridSize; c++) { if (isEnd(r, c) || isObstacle(r, c)) continue; let vExpected = 0; for (let action in actions) { let nextR = r + actions[action][0]; let nextC = c + actions[action][1]; // 撞牆或障礙物則留在原地 if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) { nextR = r; nextC = c; } let reward = -1; // 每走一步的 reward vExpected += 0.25 * (reward + gamma * V[nextR][nextC]); } newV[r][c] = vExpected; delta = Math.max(delta, Math.abs(vExpected - V[r][c])); } } V = newV; if (delta < theta) break; // 收斂 }

// 2. 找出 Greedy Policy (推導箭頭方向) let policy = Array.from({ length: gridSize }, () => Array(gridSize).fill([])); for (let r = 0; r < gridSize; r++) { for (let c = 0; c < gridSize; c++) { if (isEnd(r, c) || isObstacle(r, c)) continue; let maxVal = -Infinity; let bestActions = []; for (let action

```
in actions) {                let nextR = r + actions[action][0];                let nextC = c + actions[action][1];                                if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) {                    nextR = r;                    nextC = c;                }                                let val = -1 + gamma * V[nextR][nextC];                                // 處理浮點數誤差                if (val > maxVal + 1e-6) {                    maxVal = val;                    bestActions = [action];                } else if (Math.abs(val - maxVal) <= 1e-6) {                    bestActions.push(action); // 多個方向同等好                }            }            policy[r][c] = bestActions;        }    }

// 將 V 的數值四捨五入到小數點後兩位    let roundedV = V.map(row => row.map(val => Number(val.toFixed(2))));

// 顯示結果    document.getElementById('results').style.display = 'flex';    renderResultGrid('valueGrid', roundedV, 'value');    renderResultGrid('policyGrid', policy, 'policy'); }

// 繪製結果圖表的函數 function renderResultGrid(containerId, matrixData, type) {    const container = document.getElementById(containerId);    container.innerHTML = '';        const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

for (let i = 0; i < n; i++) {        const rowDiv = document.createElement('div');        rowDiv.className = 'grid-row';        for (let j = 0; j < n; j++) {            const cellDiv = document.createElement('div');            cellDiv.className = 'grid-cell';                const isEnd = (mapData.end[0] === i && mapData.end[1] === j);            const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j);                if (isEnd) {                cellDiv.classList.add('cell-end');                // 可自行決定終點要顯示 END 還是空白            } else if (isObstacle) {                cellDiv.classList.add('cell-obstacle');            } else {                if (type === 'value') {
  cellDiv.innerText = matrixData[i][j];                } else if (type === 'policy') {                // 將 action 陣列轉換成多個箭頭                const actionList = matrixData[i][j];                const arrowStr = actionList.map(a => arrows[a]).join('');                cellDiv.innerHTML = <span class="arrow">${arrowStr}</span>;            }        }        rowDiv.appendChild(cellDiv);        }        container.appendChild(rowDiv);    } }
```

解釋這段的目的與程式碼內容// === HW1-2: 策略評估演算法 (JavaScript 實作) ===

```
function calculatePolicy() {    const gridSize = mapData.size;    const end = mapData.end;    const obstacles = mapData.obstacles;        const theta = 1e-4;    const gamma = 1.0;        // 初始化 V 為 0    let V = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0));    const actions = {        'up': [-1, 0], 'down': [1, 0], 'left': [0, -1], 'right': [0, 1]    };

// 輔助函數：判斷是否為障礙物或終點    const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c);    const isEnd = (r, c) => end[0] === r && end[1] === c;

// 1. 執行 Policy Evaluation (隨機策略 0.25)    while (true) {        let delta = 0;        let newV = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0));        for (let r = 0; r < gridSize; r++) {            for (let c = 0; c < gridSize; c++) {                if (isEnd(r, c) || isObstacle(r, c)) continue;                let vExpected = 0;                for (let action in actions) {                    let nextR = r + actions[action][0];                    let nextC = c + actions[action][1];                                // 撞牆或障礙物則留在原地                    if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) {                        nextR = r;                        nextC = c;                    }                                let reward = -1; // 每走一步的 reward                    vExpected += 0.25 * (reward + gamma * V[nextR][nextC]);                }                newV[r][c] = vExpected;                delta = Math.max(delta, Math.abs(vExpected - V[r][c]));            }        }        V = newV;        if (delta < theta) break; // 收斂    }

// 2. 找出 Greedy Policy (推導箭頭方向)    let policy = Array.from({ length: gridSize }, () => Array(gridSize).fill([]));        for (let r = 0; r < gridSize; r++) {        for (let c = 0; c < gridSize; c++) {            if (isEnd(r, c) || isObstacle(r, c)) continue;                let maxVal = -Infinity;            let bestActions = [];                for (let action in actions) {                let nextR = r + actions[action][0];                let nextC = c + actions[action][1];                                if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) {                    nextR = r;                    nextC = c;                }                                let val = -1 + gamma * V[nextR][nextC];                                // 處理浮點數誤差                if (val > maxVal + 1e-6) {                    maxVal = val;                    bestActions = [action];                } else if (Math.abs(val - maxVal) <= 1e-6) {                    bestActions.push(action); // 多個方向同等好                }            }            policy[r][c] = bestActions;        }    }

// 將 V 的數值四捨五入到小數點後兩位    let roundedV = V.map(row => row.map(val => Number(val.toFixed(2))));

// 顯示結果    document.getElementById('results').style.display = 'flex';    renderResultGrid('valueGrid', roundedV, 'value');    renderResultGrid('policyGrid', policy, 'policy'); }

// 繪製結果圖表的函數 function renderResultGrid(containerId, matrixData, type) {    const container = document.getElementById(containerId);    container.innerHTML = '';        const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

for (let i = 0; i < n; i++) {        const rowDiv = document.createElement('div');        rowDiv.className = 'grid-row';        for (let j = 0; j < n; j++) {            const cellDiv = document.createElement('div');            cellDiv.className = 'grid-cell';                const isEnd = (mapData.end[0] === i && mapData.end[1] === j);            const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j);                if (isEnd) {                cellDiv.classList.add('cell-end');                // 可自行決定終點要顯示 END 還是空白            } else if (isObstacle) {                cellDiv.classList.add('cell-obstacle');            } else {                if (type === 'value') {
  cellDiv.innerText = matrixData[i][j];                } else if (type === 'policy') {                // 將 action 陣列轉換成多個箭頭                const actionList = matrixData[i][j];                const arrowStr = actionList.map(a => arrows[a]).join('');                cellDiv.innerHTML = <span class="arrow">${arrowStr}</span>;            }        }        rowDiv.appendChild(cellDiv);        }        container.appendChild(rowDiv);    } }
```

解釋這段的目的與程式碼內容

Gemini

這段程式碼是整個作業最核心的「大腦」與「畫筆」。它的主要目的是**利用強化學習（Reinforcement Learning）的演算法來計算出每個格子的價值，並把結果畫在網頁上**。

我可以將這段程式碼拆分成兩個主要函數來為你詳細解釋：

🧠 1. `calculatePolicy()`：計算策略與價值（核心演算法）

這個函數負責執行強化學習中的 **Policy Evaluation（策略評估）**，用來推導出數字矩陣（Value Matrix）與箭頭矩陣（Policy Matrix）。它包含了以下幾個重要階段：

- **初始化設定**：
  - `theta = 1e-4`：這是「收斂閾值」。當數值更新的幅度小於這個值時，就代表計算完成了。
  - `gamma = 1.0`：這是「折扣因子（Discount Factor）」，決定未來獎勵的重要性（1.0 代表未來與現在的獎勵一樣重要）。
  - `V`：建立一個 $n \times n$ 的陣列，將所有格子的初始價值設為 0。
- **第一階段：執行 Policy Evaluation（評估隨機策略）**：
  - 這裡使用了一個 `while (true)` 迴圈，不斷迭代更新每個格子的價值 $V(s)$，直到數值不再產生明顯變化（`delta < theta`）為止。
  - 它假設在每個格子裡，往上、下、左、右走的機率都是 **0.25**（隨機策略）。
  - **撞牆判定**：如果移動後會超出邊界或撞到障礙物，程式會讓它「留在原地」（`nextR = r; nextC = c;`）。
  - **獎勵計算**：每走一步的代價（Reward）固定為 `-1`。它利用貝爾曼期望方程式（Bellman Expectation Equation）不斷累加期望值：`vExpected += 0.25 * (reward + gamma * V[nextR][nextC])`。
- **第二階段：找出 Greedy Policy（推導最佳箭頭方向）**：
  - 當所有格子的價值 V 都算好並收斂後，程式會再掃描一次地圖。
  - 這次的目的是「看哪個方向的未來價值最高」。它會測試四個方向，找出能得到最大 `val` 的行動（Action）。

    - 為了避免 JavaScript 浮點數計算的微小誤差，這裡加上了 1e-6 的容錯範圍。如果有多個方向的數值一樣好，就會把這些方向都存進 bestActions 裡（這就是為什麼有些格子會有雙向或三向箭頭）。
- **輸出準備**：最後，它會將計算出來的 V 值四捨五入到小數點後兩位，並呼叫第二個函數來畫圖。

🎨 2. renderResultGrid()：繪製結果圖表（畫面渲染）

這個函數是一個「共用畫筆」，它的目的是**接收剛剛算出來的數學矩陣，並把它們變成網頁上肉眼可見的方格與文字**。

- **動態建立 DOM 元素**：它會根據網格大小 n，利用兩個 for 迴圈動態生成 HTML 的 標籤來組成網格。
- **狀態著色**：
    - 如果是終點，它會套用 cell-end 的 CSS 樣式（紅色背景）。
    - 如果是障礙物，套用 cell-obstacle（灰色背景）。
- **填寫內容（根據 type 決定）**：
    - 如果傳入的 type 是 'value'：它會把陣列裡的小數數字直接填入格子裡（對應 Value Matrix）。
    - 如果傳入的 type 是 'policy'：它會拿出剛剛算出的 bestActions 陣列（例如 ['up', 'right']），透過 arrows 字典把它們轉換成真正的箭頭符號（如 ↑→），並顯示在格子裡。

總結來說：這段程式碼完美取代了原本需要由 Python 後端來做的事情。它用 JavaScript 在使用者的瀏覽器裡直接把「走到終點的最短預期步數（Value）」算出來，並指出了「最聰明的走法方向（Policy）」。

這部分的邏輯比較吃重數學基礎。**你對程式碼裡的「貝爾曼方程式計算（vExpected）」或是「邊界撞擊處理」有哪裡需要我進一步舉例說明的嗎？**

You

寫一個 readme.md 內容包含 demo網址 https://bibaisafish.github.io/DRL_HW1/

Gemini

這是一份為你的專案量身打造的 README.md 檔案。內容涵蓋了作業要求、功能介紹、操作步驟，並且把你的 Demo 網址放在了最顯眼的位置，讓教授或助教點進去就能馬上看到成品！

你可以直接複製以下內容，貼到你的 README.md 檔案中：

```
# Deep Reinforcement Learning - HW1: Grid World Policy Evaluation

這是一個基於純前端技術 (HTML, CSS, JavaScript) 開發的「網格世界 (Grid World)」強化學習互動式網頁應用程式。本專案實作了隨機策略下的**策略評估 (Poli

👉 **[點此觀看 Live Demo](https://bibaisafish.github.io/DRL_HW1/)**

---

## 🌟 專案功能 (Features)

本專案分為兩個主要部分，完全符合 HW1 的作業要求：

### HW1-1: 網格地圖開發 (前端互動)
* **動態網格生成**：允許用戶自由輸入維度 `n` (範圍從 5 到 9)，動態生成 `n x n` 的網格地圖。
* **狀態機互動設計**：透過流暢的滑鼠點擊流程設定環境：
  1. **起點 (Start)**：點擊設定為綠色單元格。
  2. **終點 (End)**：點擊設定為紅色單元格。
  3. **障礙物 (Obstacles)**：設定 `n-2` 個障礙物，點擊變為灰色單元格（保留原始數字顯示）。

### HW1-2: 策略顯示與價值評估 (核心演算法)
* **策略評估 (Policy Evaluation)**：基於隨機策略（上、下、左、右機率各 25%），透過貝爾曼期望方程式 (Bellman Expectation Equation) 迭代計算出每個狀
* **策略矩陣 (Policy Matrix)**：根據算出的 `V(s)` 推導出 Greedy Policy，在畫面上顯示每個單元格的最佳行動方向（↑, ↓, ←, →），若有多個最佳方向亦會同
* **價值矩陣 (Value Matrix)**：清楚呈現每個單元格最終收斂的預期價值（四捨五入至小數點後兩位）。

---

## 🛠️ 技術棧 (Tech Stack)

* **HTML5**：架構網頁主體與矩陣容器。
* **CSS3**：使用 Flexbox 排版，設計直觀的 UI 介面與狀態顏色。
* **Vanilla JavaScript (ES6)**：
  * 處理 DOM 操作與滑鼠點擊狀態機。
  * 實作 Reinforcement Learning 的 Policy Evaluation 演算法（無須後端伺服器即可在瀏覽器端完成矩陣運算）。

---

## 🚀 如何使用 (How to Use)

1. 開啟 [Demo 網頁](https://bibaisafish.github.io/DRL_HW1/)。
2. 在左上角的輸入框輸入 5 到 9 之間的數字，並點擊 **"Generate Square"** 生成網格。
3. 根據畫面上方的提示，依序點擊網格來設定：
   * 第 1 下：設定**起點** (綠色)。
   * 第 2 下：設定**終點** (紅色)。
   * 接下來的 `n-2` 下：設定**障礙物** (灰色)。
4. 設定完成後，點擊下方出現的 **"計算策略與價值 (Calculate)"** 按鈕。
5. 頁面下方會立刻生成並顯示計算完成的 **Value Matrix** (數值) 與 **Policy Matrix** (方向箭頭)。
```

```
---

## 📌 演算法參數設定 (Hyperparameters)

* **Reward (獎勵)**：每移動一步的 reward 為 `-1`（目標為尋找最短路徑）。
* **Discount Factor ($\gamma$)**：`1.0`
* **Convergence Threshold ($\theta$)**：`1e-4`
* **Policy**：隨機策略（Random Policy），每個方向的轉移機率為 `0.25`。撞牆或撞障礙物則狀態留在原地。
```

這份 README 結構清晰，而且將技術亮點（無後端、純 JS 運算）和演算法參數都標示了出來，這在交作業時通常能讓評分者一目了然。

你覺得這個 README 的排版和內容可以嗎？需要我教你怎麼用 Git 把這些檔案推送到 GitHub Pages 上嗎？